

MIDTERM 2 NOTES

Login Openshift Web Console: `oc login -u developer -p developer`

<https://api.ocp4.example.com:6443>

COMMAND: `oc login -u <USERNAME> -p <PASSWORD> <API_ENDPOINT>:<PORT>`

Admin user: `-u admin -p redhatocp`

After, access web console using: `oc whoami --show-console`

Terminal is located at Compute > Nodes > {node} > Terminal

To enable host binaries on the node: `chroot /host`

To view the status of the kubelet node agent service: `systemctl status kubelet`

To exit a running command press Q

To view the status of the CRI-O container runtime interface: `systemctl status crio`

To install kubectl: `curl -LO https://dl.k8s.io/release/\$\(curl -L -s https://dl.k8s.io/release/stable.txt\)/bin/linux/amd64/kubectl`

`curl -LO https://dl.k8s.io/\$\(curl -L -s https://dl.k8s.io/release/stable.txt\)/bin/linux/amd64/kubectl.sha256`

To check if kubectl installed: `echo "$(cat kubectl.sha256) kubectl" | sha256sum --check`

Install kubectl CLI: `sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl`

Get information about resources: `kubectl explain pod`

Create a new project: `oc new-project PROJECT_NAME`

Select a project to work on: `oc project PROJECT_NAME`

Get control plane link: `oc cluster-info`

Cluster resources API version on the server: `oc api-versions`

To get list of resources: `oc get RESOURCETYPE1, RESOURCETYPE2, RESOURCETYPEn`

****add -o json to receive them formatted as JSON**

****add -o yaml to receive them formatted as YAML**

****add -n NAMESPACE_NAME to specify namespace**

****add --show-labels to see the labels used by the pods**

****add -o wide to see which selector uses resource**

Get all most important resources: `oc get all`

Get details of specific resource: `oc get RESOURCE_TYPE RESOURCE_NAME`

Get info about API resource: `oc explain PATH.TO.API.RESOURCE`

*****add --recursive to get all fields without descriptions***

Create resources from .yaml file: `oc create -f FILE_NAME.yaml`

Get high level overview of the project: `oc status`

*****add --suggest to get additional details of issues***

To delete resources: `oc delete RESOURCE_NAME RESOURCE_TYPE`

To specify the namespace where you want to execute the command you need to mention it after the command: `oc get pods --namespace NAMESPACE_NAME`

To view Kubernetes resources; `oc api-resources`

*****add --namespaced=true to return namespaced ones***

*****add --api-group GROUP_NAME to show only ones on the specified api group***

*****add --sort-by NAME to order them sorted by specified field***

To view more details about resource: `oc describe RESOURCE_NAME APP_NAME`

OPERATORS

List all cluster operators: `oc get clusteroperators`

*****add dns -o yaml to get it formatted as yaml***

To get a human readable description for a cluster use `describe clusteroperators NAME_OP`

To get the add-on operators use `oc get operators`

To view resource usage we use `oc adm top`

*****add --sum to print the sum of resource usages***

*****add -A to display for all namespaces***

*****add -n NAMESPACE_NAME to display for specific namespace***

*****add --containers to display for the containers within a pod***

To print the events use `oc get events`

*****add --sort-by .metadata.creationTimestamp to order by time***

To display logs of processes on a node: `oc adm nlogs NODE_NAME`

*****add --role ROLE to filter with specified role***

*****add -u UNIT to filter with specified unit***

*****add --path=PATH to filter with specified process***

*****add --tail=X to limit output to the last X entries***

To start a debug session use `oc debug node/NODE_NAME`

*****use exit to exit the session***

To display details about the status of a service use `systemctl status SERVICE_NAME`

CREATE LINUX CONTAINERS AND KUBERNETES PODS

Create and deploy an application in a pod: `oc run RESOURCE/NAME --image IMAGE`

*****use double dashes to add arguments: -- arg1 arg2 argN***

*****add --command -- cmd arg1 arg2 to add custom command***

*****add -it before the podname to start an interactive, remote shell in a container***

*****add -n NAMESPACE_NAME to specify the namespace***

*****manage restart policy by adding --restart [Always / OnFailure / Never]***

*****add --rm to automatically delete on exit***

*****add --env NAME=VALUE to add environment variables***

To get description of the app: `oc describe project APP_NAME`

To execute a command in a running container: `oc exec RESOURCE/NAME -- COMMAND`

*****add -c CONTAINER to specify which container is to execute the command***

To print the recent log lines use: `oc logs APPNAME`

*****add --selector=" to filter based on key:value***

*****add --tail= to specify number of lines to be displayed***

*****add -c CONTAINER_NAME to print for specific container***

*****add -p to print for the previous container instance in the pod, if it exists.***

To start an interactive pod on a running container use `oc attach CONTAINER -it`

To delete a pod use: `oc delete pod PODNAME`

*****add -l KEY:VALUE to select resources based on label***

*****add -f PATH/TO/FILE to use a JSON or YAML that has the name of the resource***

*****add --grace-period=X to give the pod X time to finish its processes***

*****add --now to set that time to 1 second***

*****add --force to force its deletion***

*****add --all to delete all pods***

To delete a project: `oc delete project PROJECT_NAME`

To inspect images: *oc image info IMAGE*

To add layers to an image: *oc image append*

To copy image to local disk: *oc image extract*

To copy or mirror images: *oc image mirror*

Generate resource definitions from .yaml file: *oc process*

*****add --parameteres to have the parameters displayed***

To create a new application: *oc new-app*

*****add --template TEMPLATE to create it from a template***

*****add --param KEY=VALUE to add parameters***

*****add --file=./file/path/example.yaml***

*****add -l key=value to add labels***

Create a service for a deployment: *oc expose deployment/deployment-name*

*****add --selector to specify label selectors***

*****add --port XXXX to specify the port it listens on***

*****add --target-port XXXX to specify the port used for communication***

*****add--protocol to determine the network protocol, default is TCP***

*****add --name to name the service***

To pause rollout: *oc rollout pause deployment/NAME*

To resume rollout: *oc rollout resume deployment/NAME*

To check rollout process: *oc rollout status depeloyment/NAME*

To undo rollout: *oc rollout undo deployment/NAME*

Command	Description
<code>crictl pods</code>	List all pods on a node.
<code>crictl image</code>	List all images on a node.
<code>crictl inspect</code>	Retrieve the status of one or more containers.
<code>crictl exec</code>	Run a command in a running container.

Command	Description
<code>crictl logs</code>	Retrieve the logs of a container.
<code>crictl ps</code>	List running containers on a node.

Inspecting and Managing Container Images

To install skopeo: `sudo dnf -y install skopeo`

To login with skopeo: `skopeo login quay.io`

List all available tags on image: `skopeo list-tags IMAGE`

View detailed information for image: `skopeo inspect TRANSPORT://IMAGE`

*****add --config to view configuration, metadata and history of repository**

To copy image somewhere: `skopeo copy TRANSPORT://FROM TRANSPORT://TO`

To delete image: `skopeo delete TRANSPORT://IMAGE`

Sync images from location to another: `skopeo sync --src TRANSPORT --dest TRANSPORT SOURCEIMAGE DESTINATIONIMAGE`

Check logged in credentials: `cat ${XDG_RUNTIME_DIR}/containers/auth.json`

Use these tools to validate the functions and environment for a running container:

- `oc describe`: Display the details of a resource.
- `oc edit`: Edit a resource configuration by using the system editor.
- `oc patch`: Update a specific attribute or field for a resource.
- `oc cp`: Copy files and directories to and from containers.
- `oc exec`: Execute a command within a specified container.
- `oc port-forward`: Configure a port forwarder for a specified container.
- `oc logs`: Retrieve the logs for a specified container.
- `oc rsync`: Synchronize files and directories to and from containers.
- `oc rsh`: Start a remote shell within a specified container.

DEFINITIONS

A pod is the smallest compute unit that you can define, deploy, and manage in RHOCP. A pod runs one or more containers that represent a single application. Containers in the pod share resources, such as networking and storage.

A deployment describes the intended state of a component of the application as a pod template. A deployment manages one or more replica sets.

RHOCP adds projects to enhance the function of Kubernetes namespaces. A project is a Kubernetes namespace with additional annotations, and is the primary method for managing access to resources for regular users. Projects can be created from templates and must use RBAC for organization and permission management. Administrators must grant cluster users access to a project. If a cluster user is allowed to create projects, then the user automatically has access to their created projects.

Projects provide logical and organizational isolation to separate your application component resources. Resources in one project can access resources in other projects, but not by default.

Services: You can configure internal pod-to-pod network communication in RHOCP by using a service. Applications send requests to the service name and port. RHOCP provides a virtual network, which reroutes such requests to the pods that the service targets by using labels.

RHOCP uses the Kubernetes persistent volume (PV) framework to enable cluster administrators to provision persistent storage for a cluster. Developers can use persistent volume claims (PVCs) to request PV resources without having specific knowledge of the underlying storage infrastructure. After a PV is bound to a PVC, that PV cannot then be bound to additional PVCs. Because PVCs are namespaced objects and PVs are globally scoped objects, this binding effectively scopes a bound PV to a single namespace until the binding PVC is deleted.

Secrets provide a mechanism to hold sensitive information, such as passwords, private source repository credentials, sensitive configuration files, and credentials to an external resource, such as an SSH key or an OAuth token. You can mount secrets into containers by

using a volume plug-in. Kubernetes can also use secrets to perform actions, such as on pods, declaring environment variables. Secrets can store any type of data. Kubernetes and OpenShift support different types of secrets, such as service account tokens, SSH keys, and TLS certificates.

SPECIFIC

To create a network route: *oc expose service api-frontend --hostname api.apps.acme.example.com --target-port 31234*

To create an Ingress object: *oc create ingress ingr-sakila --rule="ingr-sakila.apps.ocp4.example.com/*=sakila-service:8080"*

To configure session persistence for an Ingress object by using a cookie, use the following command:

```
[user@host ~]$ oc annotate ingress ingr-example ingress.kubernetes.io/affinity=cookie
```

To configure session persistence for a Route object with a custom cookie named myapp, use the following command:

```
[user@host ~]$ oc annotate route route-example router.openshift.io/cookie_name=myapp
```

To capture the route hostname, use the following command:

```
[user@host ~]$ ROUTE_NAME=$(oc get route route-example -o jsonpath='{.spec.host}')
```

To access the route and save the cookie, use the following command:

```
[user@host ~]$ curl $ROUTE_NAME -k -c /tmp/cookie_jar
```

To use the saved cookie for subsequent requests, use the following command:

```
[user@host ~]$ curl $ROUTE_NAME -k -b /tmp/cookie_jar
```

To scale a deployment, use the following command:

```
[user@host ~]$ oc scale --replicas=5 deployment/scale
```

Create a generic secret that contains key-value pairs from literal values that are typed on the command line:

```
[user@host ~]$ oc create secret generic secret_name \  
--from-literal key1=secret1 \  
--from-literal key2=secret2
```

Create a generic secret by using key names that are specified on the command line and values from files:

```
[user@host ~]$ kubectl create secret generic ssh-keys \  
--from-file id_rsa=/path-to/id_rsa \  
--from-file id_rsa.pub=/path-to/id_rsa.pub
```

Create a TLS secret that specifies a certificate and the associated key:

```
[user@host ~]$ oc create secret tls secret-tls \  
--cert /path-to-certificate --key /path-to-key
```

The syntax for creating a configuration map and for creating a secret closely match. You can enter key-value pairs on the command line, or use the content of a file as the value of a specified key. You can use either the `oc` or `kubectl` command-line tools to create a configuration map. The following command shows how to create a configuration map:

```
[user@host ~]$ kubectl create configmap my-config \  
--from-literal key1=config1 --from-literal key2=config2
```

You can also use the `cm` shortname to create a configuration map.

```
[user@host ~]$ oc create cm my-config \  
--from-literal key1=config1 --from-literal key2=config2
```


he following command creates a generic secret that contains key-value pairs from literal values that are typed on the command line: user with the demo-user value, and root_password with the zT1kTgk value.

```
[user@host ~]$ oc create secret generic demo-secret \
--from-literal user=demo-user \
--from-literal root_password=zT1kTgk
```

You can also create a generic secret by specifying key names on the command line and values from files:

```
[user@host ~]$ oc create secret generic demo-secret \
--from-file user=/tmp/demo/user \
--from-file root_password=/tmp/demo/root_password
```

You can mount a secret to a directory within a pod. Kubernetes creates a file for each key in the secret that uses the name of the key. The content of each file is the decoded value of the secret. The following command shows how to mount secrets in a pod:

```
[user@host ~]$ oc set volume deployment/demo \
--add --type secret \
--secret-name demo-secret \
--mount-path /app-secrets
```